

10 — CAP + AI Core Integration + UI5 Frontend

What it is: the other half of sheet 09 — how the CAP service layer actually calls SAP AI Core, and how a SAPUI5 app calls CAP. This closes the loop: **UI5 → CAP OData/action → AI Core → HANA → back to UI5.**

End-to-End Call Chain

UI5 Controller (JS)

```
--calls OData action--> CAP Service (runtime.js)
--calls--> AI Core (Orchestration Service, sheet 07/08 concepts)
--reads/writes--> HANA (CQL, sheet 09)
<--returns RagResponse--
<--updates JSONModel / OData model--
```

CAP Service Implementation Calling AI Core (srv/runtime.js)

```
module.exports = cds.service.impl(async function () {
  const capllmplugin = await cds.connect.to("cap-llm-plugin"); // destination-backed AI Core connector

  this.on("getChatRagResponse", async (req) => {
    const { user_query, conversationId } = req.data;

    // 1. embed the query (equivalent of sheet 04's embedding_setup.py)
    const embeddingConfig = cds.env.requires["gen-ai-hub"]["text-embedding-ada-002"];
    const embedding = await capllmplugin.getEmbeddingWithConfig(embeddingConfig, user_query);

    // 2. similarity search in HANA (equivalent of sheet 04's similarity_search.py)
    const chunks = await capllmplugin.similaritySearch(
      "DocumentChunk", "embedding", "text_chunk", embedding, "COSINE_SIMILARITY", 5);

    // 3. call the LLM with retrieved context (equivalent of sheet 05's rag_pipeline.py)
    const chatResponse = await capllmplugin.getHarmonizedChatCompletion({
      clientConfig: chatCompletionConfig, getContent: true,
    });

    // 4. persist via CQL (sheet 09)
    await INSERT.into(Message).entries({ cID: conversationId, content: chatResponse, role: "assistant" });

    return { message: chatResponse, sources: chunks };
  });
});
```

AI Core Destination Config (package.json)

```
"cds": { "requires": { "gen-ai-hub": {
  "kind": "GenAIHubDestination",
  "destination": "aicore-destination" // points to a BTP Destination (OAuth2ClientCredentials)
}}}
```

This is the CAP-world equivalent of sheet 01/02's .env + AICORE_CLIENT_ID/SECRET — instead of raw env vars, credentials live in a BTP **Destination** the platform resolves at runtime.

UI5 App Wiring (app/chat-ui/webapp/)

```
webapp/
├─ manifest.json      <- declares dataSources: mainService -> /odata/v4/chat/
├─ Component.js
├─ controller/
│   └─ App.controller.js
└─ view/
    └─ App.view.xml
```

```
// manifest.json (excerpt)
"dataSources": { "mainService": { "uri": "/odata/v4/chat/", "type": "OData", "settings": {"odataVersion": "4"} } }

// App.controller.js – calling the CAP action from UI5
onSend: function () {
  const oModel = this.getView().getModel(); // the OData V4 model from manifest.json
  const oAction = oModel.bindContext("/getChatRagResponse(...)", null, {
    $$updateGroupId: "submitLater",
  });
  oAction.setParameter("user_query", this._oInputField.getValue());
  oAction.execute().then((oResult) => {
    this._appendMessageToChat(oResult.getObject()); // update the chat view
  });
},
```

Deployment — mta.yaml (Multi-Target Application)

```
modules:
- name: my-app-srv # CAP Node.js service
  type: nodejs
- name: my-app-approuter # serves the UI5 app, routes to srv, handles auth
  type: approuter.nodejs
- name: my-app-db-deployer # deploys db/schema.cds to the HDI container
  type: hdb
resources:
- name: my-app-xsuaa # auth – from xs-security.json
- name: my-app-destination # AI Core / other external system credentials
- name: my-app-hdi # the HANA HDI container itself
```

Key Points

- The **approuter** is the single entry point in production — it serves the UI5 static files AND proxies /odata/... calls to the CAP service, attaching the user's XSUAA JWT.
- AI Core credentials live in a **BTP Destination**, not a .env file — same idea as sheets 01/02's env vars, just resolved by the platform instead of python-dotenv.
- The UI5 side only ever calls **your own CAP service** — it never talks to SAP AI Core directly, keeping API credentials fully server-side (same principle noted in sheet 09).
- bindContext(...).execute() is how UI5 invokes an OData V4 **action** (as opposed to reading an entity set) — this is the UI5 equivalent of Python's agent.ask(question) call pattern.
- Local development uses a [hybrid] MTA profile with mocked auth; [production] binds real XSUAA — lets you test the CAP+AI Core logic without deploying first.

Interview Q&A

Q: Why route the UI5 app's AI calls through CAP instead of calling AI Core directly from the browser? A: Keeps AI Core credentials/deployment IDs server-side, lets you apply masking/filtering/auth/rate-limiting centrally, and avoids exposing internal service topology to the client.

Q: What is the approuter's role in this architecture? A: Single entry point that serves static UI5 files, reverse-proxies OData calls to the CAP service, and enforces XSUAA authentication — the client never talks to srv directly.

Q: How does this project structure map back to the plain-Python templates in sheets 01-08? A: Same layers, different language/runtime — db/schema.cds = the vector table DDL (sheet 04), runtime.js's AI Core calls = llm_setup.py/rag_pipeline.py (sheets 02/05), CQL = hdbcli SQL (sheet 01/03), and the UI5 controller = the CLI runner/Streamlit app (sheet 06) as the presentation layer.